OUTAGE DISSECTIONS • BLAST RADIUS CONTAINMENT • HIGH-SCALE RELIABILITY

Dissecting GitHub Outage: Why Failure Domains Must Be Localized

Curated from Arpit Bhayani's Architecture Dissection Series • Core Theme: Blast Radius Engineering

EXECUTIVE ARCHITECTURAL ABSTRACT

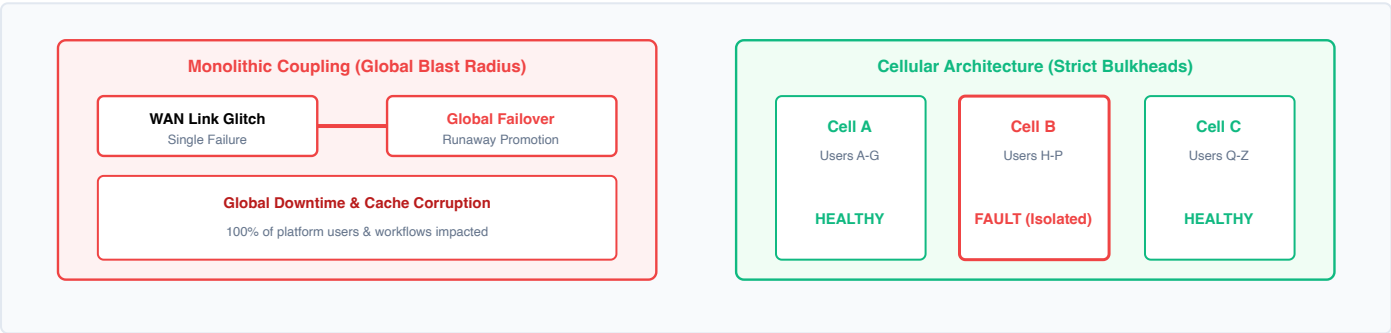
High availability is defined not by avoiding failure, but by **containing its blast radius**. In GitHub's major infrastructure outages, localized hardware or network glitches escalated into platform-wide blackouts because shared dependencies, telemetry agents, and automated failover control planes spanned regional boundaries. Mitigating these systemic vulnerabilities requires adopting **cellular architectures**, enforcing **bulkheads**, and decoupling the control plane from the operational data plane.

1. THE FALLACY OF GLOBAL MONOLITHIC COUPLING

When services share unpartitioned global dependencies, any local perturbation naturally ripples outward:

- **Cascading Cross-Region Failover:** An isolated 43-second packet loss event on a transatlantic fiber cable triggered automated master promotions across the opposite coast, causing platform-wide split-brain.
- **Unbounded Blast Radius:** Because all users were hosted on shared global MySQL topologies, 100% of the developer user base suffered downtime from a fault restricted to a single network segment.
- **The Feedback Loop of Chaos:** As web servers encountered slow responses, client retry storms overwhelmed surviving database nodes, converting a minor link drop into an unrecoverable saturation cascade.

2. VISUALIZING BLAST RADIUS CONTAINMENT



3. THE TELEMETRY BLINDNESS PARADOX

A fatal vulnerability in large architectures is when the observer depends on the observed:

Circular Heartbeat Dependencies

Orchestrator monitoring daemons relied on MySQL database connections to write their own cluster health logs. When proxy layers saturated during the link blip, telemetry daemons hung. The control plane interpreted its own local connection starvation as remote cluster death.

False Automation Runaway

Because telemetry was degraded, automated failover heuristics executed in the dark. Without positive quorum confirmation from both datacenters, the system acted on partial network data, promoting replicas prematurely and initiating split-brain.

4. ARCHITECTURAL PILLARS OF FAILURE DOMAIN LOCALIZATION

Localizing failures requires engineering hard boundaries across three orthogonal dimensions:

Pillar 1: Cellular Topologies (Siloing Compute & Storage)

Divide the platform into N independent, complete stacks (cells). Each cell owns its application workers, cache layer, background queues, and database shards. If a bug or link disruption strikes Cell K, only 1/N of the user population is impacted. The remaining N - 1 cells operate with zero degradation.

Pillar 2: Resource Bulkhead Isolation

Isolate critical physical resources so non-critical background traffic cannot exhaust interactive paths:

- **Dedicated Connection Pools:** Separate user-facing HTTP request pools from background webhook/Actions pools.
- **Isolated Redis Instances:** Never co-locate session authentication caches with transient background queue data.
- **Out-of-Band Control Networks:** Control plane heartbeats must travel over dedicated management networks, never competing with user data plane traffic.

5. CONTROL PLANE DECOUPLING BLUEPRINT

The control plane (topology managers, load balancer controllers, DNS failover engines) must be strictly decoupled from the data plane:

```
# Architectural Invariant: Control Plane Independence
class ClusterTopologyManager:
    def __init__(self, consensus_quorum, local_storage):
        self.consensus = consensus_quorum  # Independent Raft/etcd cluster
        self.storage = local_storage        # Local out-of-band state store

    def evaluate_failover(self, target_node):
        # 1. Require strict multi-region quorum witness
        if not self.consensus.has_majority_witness():
            raise QuorumUnavailableError("Cannot failover: No multi-region quorum witness")

        # 2. Positive Fencing Verification
        fence_ok = self.consensus.verify_fencing_token(target_node.current_epoch)
        if not fence_ok:
            raise FencingError("Cannot promote: Previous master is not positively fenced")

        # 3. Safe local promotion within bounded failure domain
        return self.promote_local_replica(target_node)
```

6. MONOLITH VS. CELLULAR ARCHITECTURE COMPARISON

Architecture Characteristic	Global Monolithic Architecture	Cellular Distributed Architecture
Blast Radius of Database Bug	**100% of Platform**	Strictly bounded to 1/N (e.g., 5% in 20 cells)
Cross-Region Network Blip Impact	Global failovers, split brain, cache pollution	Isolated to inter-cell replication; no failover cascade
Control Plane Coupling	Shared database & shared telemetry tier	Independent out-of-band consensus per cell
Rollout & Deployment Risk	Binary upgrade affects entire platform	Canary rollout deployed 1 cell at a time

7. PRODUCTION ENGINEERING TAKEAWAYS

1. **Prove Your Failure Boundaries in Production:** A failure boundary that has not been validated via chaos engineering (e.g. cutting regional links under peak load) does not exist. It is merely an unverified theoretical hope.
2. **Control Systems Must Outlive Data Systems:** The systems responsible for diagnosing and healing outages must never share memory, databases, or network pipes with the systems being monitored.
3. **Fail Locally, Recover Locally:** Avoid automated WAN migrations during transient faults. Confining remediation within a single local Availability Zone maintains sub-second recovery times without risking catastrophic cross-region split brain.